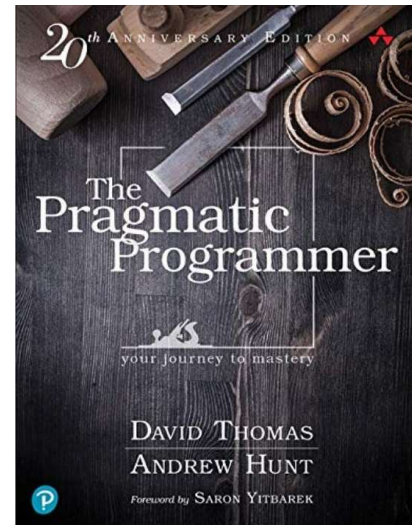


Entwurfsprinzipien

Prinzipien und Paradigmen der OOP

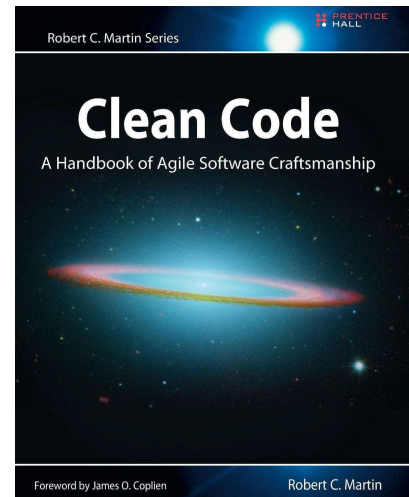
Buchempfehlung 1

- Klassiker in SW-Entwicklung
- Vermittelt wichtige Prinzipien und Techniken
- Fördert pragmatische Ansätze für Lösungen
- Behandelt Themen wie Code-Qualität, Debugging und Testing
- Bietet praktische Tipps, die direkt in der Arbeit angewendet werden können



Buchempfehlung 2

- Standardwerk von Robert C. Martin
- Detaillierte Anleitungen zum Schreiben von sauberem und wartbarem Code
- Fokus auf **Lesbarkeit** von Code
- Best Practices und Beispiele für leicht zu lesenden, debuggenden und modifizierenden Code
- Auslöser für **Clean Code Developer**



Entwurfsprinzipien der OOP

- Trenne Veränderliches von Konstantem.
- Programmiere gegen eine Schnittstelle (d. h. Abstraktion) statt auf eine Implementierung.
- Ziehe die Komposition der Vererbung vor.
- Strebe nach lockerer Bindung zwischen Klassen und hohe Kohäsion in Klassen an.

<https://infoskript.de/entwurfsprinzipien>

Trenne Veränderliches von Konstantem

Manche Teile einer Software werden sich wahrscheinlich mit der Zeit ändern, andere eher nicht. Damit nicht das ganze System überarbeitet werden muss, sollten solche Teile voneinander getrennt werden.

Programmiere auf eine Schnittstelle statt auf eine Implementierung

Benutze Obertypen, um bei späteren Änderungen flexibel zu sein. Schnittstelle ist nicht unbedingt gleichbedeutend mit "Interface", sondern soll als "Obertyp" interpretiert werden.

Ziehe Komposition der Vererbung vor

Anstatt komplexe Vererbungshierarchien zu entwickeln, kann man auch Funktionalität über Komposition erweitern.

Kopplung und Kohäsion

Strebe eine geringe Kopplung zwischen Klassen und eine hohe Kohäsion in Klassen an.

Weitere Prinzipien mit Akronymen

- KISS Keep It Simple, Stupid
- DRY Don't Repeat Yourself
- YAGNI You Ain't Gonna Need It
- SOC Separation of concerns
- SOLID SRP, OCP, LSP, ISP, DIP

Mehr Prinzipien auf
Clean Code Developer:
[Die Tugenden](#)

Kennt ihr Prinzipien?

Was haltet ihr von Ihnen?

CCD Website anschauen:

- Ich weiß ja nicht, aber bei der Menge an Prinzipien
fühle ich mich überfordert.

Anekdote von Bill Gates über faule Entwickler

SOLID Prinzipien

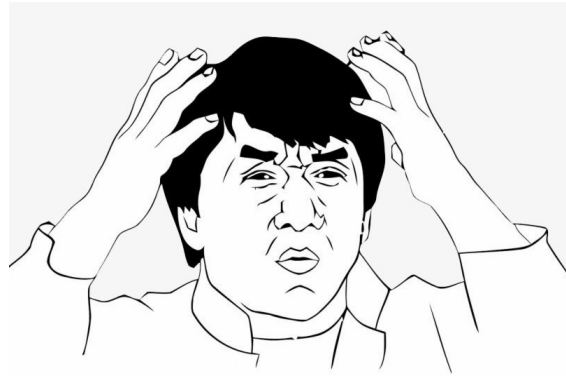
SOLID-Prinzipien auf die richtige Art

<https://www.youtube.com/watch?v=V3TUEeB0kW0&t=90>

Wir schauen uns SOLID an, weil es auf der Agenda steht.

Was sind SOLID Prinzipien?

- Werden häufig zitiert
- Selten praktiziert
- Und noch seltener verstanden



Für mich klingen sie irgendwo im Hinterkopf als vage Richtlinien und tragen zu meinem stärker werdenden Imposter-Syndrom bei.

Also lasst uns diese Buzzwords aufdröseln und schauen, wie sie in realen Szenarien angewendet werden können.

[jacky chan confused meme]

SOLID



SRP – Single Responsibility Principle



OCP – Open/Closed Principle



LSP – Liskovs Substitution Principle



ISP – Interface Segregation Principle



DIP – Dependency Inversion Principle

Single Responsibility Prinzip

*Es sollte nie ehr als einen Grund geben,
eine Klasse zu modifizieren.*

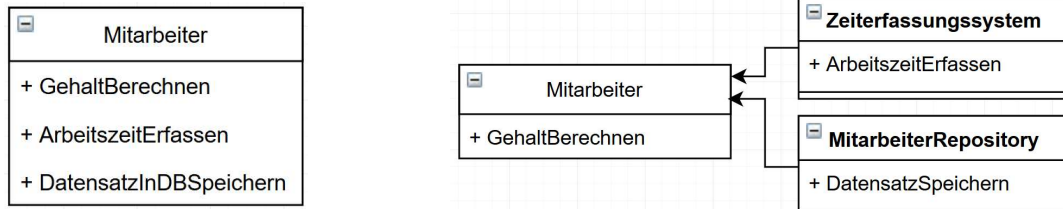
Just because you can doesn't mean you should



Single-Responsibility-Prinzip

“There should never be more than one reason for a class to change.”

– Robert C. Martin: Agile Software Development: Principles, Patterns, and Practices



Das Single-Responsibility-Prinzip besagt, dass jede Klasse nur eine einzige Verantwortung haben sollte. Verantwortung wird hierbei als „Grund zur Änderung“ definiert.

Mehr als eine Verantwortung für eine Klasse führt zu mehreren Bereichen, in denen zukünftige Änderungen notwendig werden können. Die Wahrscheinlichkeit, dass die Klasse zu einem späteren Zeitpunkt geändert werden muss, steigt zusammen mit dem Risiko, sich bei solchen Änderungen subtile Fehler einzuhandeln. Dieses Prinzip führt in der Regel zu Klassen mit hoher [Kohäsion](#), in denen alle Methoden einen starken gemeinsamen Bezug haben.

Open Closed Prinzip

Eine Softwareentität sollte offen für Erweiterungen,
aber zugleich auch geschlossen gegenüber Modifikationen sein.

Open chest surgery is not needed when putting on a coat



Open-Closed-Prinzip

“Modules should be both open for extension and closed for modification.”

– Bertrand Meyer: Object Oriented Software Construction

```
static void Main()
{
    ArrayList list = new ArrayList();
    list.Add(new Kreis { Formtyp = GeometrischeForm.Kreis, Radius = 12 });
    list.Add(new Quadrat { Formtyp = GeometrischeForm.Quadrat, Seitenlänge = 21 });

    foreach (Grafik item in list)
    {
        switch(item.Formtyp)
        {
            case GeometrischeForm.Quadrat:
                break; // Logik für das Quadrat
            case GeometrischeForm.Kreis:
                break; // Logik für den Kreis
        }
    }
}
```

Das Open-Closed-Prinzip besagt, dass Software-Einheiten (hier [Module](#), [Klassen](#), [Methoden](#) usw.) Erweiterungen möglich machen sollen (dafür offen sein), aber ohne dabei ihr Verhalten zu ändern (ihr Sourcecode und ihre Schnittstelle sollte sich nicht ändern).

Eine Erweiterung im Sinne des Open-Closed-Prinzips ist beispielsweise die [Vererbung](#).

Diese verändert das vorhandene Verhalten einer Klasse nicht, erweitert sie aber um zusätzliche Funktionen oder Daten.

Überschriebene Methoden verändern auch nicht das Verhalten der Basisklasse, sondern nur das der abgeleiteten Klasse.

Open-Closed-Prinzip

“Modules should be both open for extension and closed for modification.”

– Bertrand Meyer: Object Oriented Software Construction

```
static void Main()
{
    List<Grafik> list = new List<Grafik>();
    list.Add(new Kreis {Radius = 12 });
    list.Add(new Quadrat {Seitenlänge = 21 });

    foreach (Grafik item in list)
    {
        item.Zeichnen();
    }
}
```

In Theorie:

Code vor Änderungen schützen

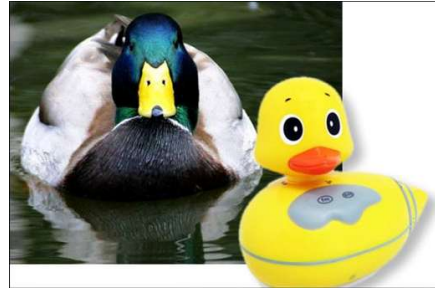
In der Praxis:

Schmerzpunkte bei Vererbung

Tendenz für Overengineering und zu viele Vererbungen!

Liskov'sches Substitutions- prinzip

If it looks like a duck, quacks like a duck, but needs batteries - you probably have the wrong abstraction



Wenn für jedes Objekt $o1$ vom Typ S ein Objekt $o2$ vom Typ T existiert, sodass für alle Programme P , die in T definiert sind, das Verhalten von P unverändert bleibt, wenn $o1$ für $o2$ substituiert wird, dann ist S ein Subtyp von T .

In anderen Worten:

Objekte einer Basisklasse sollten durch Objekte einer Unterklasse ersetzbar sein, ohne dabei das Fehlverhalten entsteht.

Wenn also die Unterklasse das Verhalten der Basisklasse bricht, ist etwas schief gelaufen.

Liskovsches-Substitutionsprinzip

„Let $q(x)$ be a property provable about objects x of type T . Then $q(y)$ should be true for object y of type S where S is a subtype of T .“

– Barbara H. Liskov, Jeannette M. Wing: Behavioral Subtyping Using Invariants and Constraints

```
class Rechteck
{
    protected int höhe;
    protected int breite;
    public virtual void SetHöhe(int wert)
    {
        höhe = wert;
    }
    public virtual void SetBreite(int wert)
    {
        breite = wert;
    }
}
```

```
class Quadrat : Rechteck
{
    public override void SetHöhe(int wert)
    {
        höhe = wert;
        breite = wert;
    }
    public override void SetBreite(int wert)
    {
        höhe = wert;
        breite = wert;
    }
}
```

Logische Folgerung aus OCP: Nicht das Verhalten, sondern nur die Algorithmen werden anpassbar.

Das Liskovsches Substitutionsprinzip (LSP) oder *Ersetzbarkeitsprinzip* fordert, dass eine Instanz einer [abgeleiteten Klasse](#) sich so verhalten muss, dass jemand, der meint, ein Objekt der [Basisklasse](#) vor sich zu haben, nicht durch unerwartetes Verhalten überrascht wird, wenn es sich dabei tatsächlich um ein Objekt eines Subtyps handelt.

Häufig ist eine Verletzung des Prinzips nicht auf den ersten Blick offensichtlich.

Interface Segregation Prinzip

*Konsumenten sollten nur die Methoden implementieren
müssen, welche sie wirklich benötigen.*

You want me to plug this in, where?



Interface-Segregation-Prinzip

“Clients should not be forced to depend upon interfaces that they do not use.”

– Robert C. Martin: The Interface Segregation Principle



```
public class Penguin : IBird {  
    public void Walk() {  
        Console.WriteLine("Walk!");  
    }  
    public void Fly() {  
        throw new NotImplementedException();  
    }  
}  
  
public interface IVolant {  
    void Fly();  
}
```

Das Interface-Segregation-Prinzip dient dazu, zu große Interfaces aufzuteilen. Die Aufteilung soll gemäß den Anforderungen der Clients an die Interfaces gemacht werden - und zwar derart, dass die neuen Interfaces genau auf die Anforderungen der einzelnen Clients passen. Die Clients müssen also nur mit Interfaces agieren, die das, und nur das können, was die Clients benötigen.

Vermeide

Dependency Inversion Prinzip

*Es ist besser sich auf Abstraktionen statt konkreter
Implementierungen zu verlassen.*

Would you solder a lamp directly to the electrical wiring in a wall?



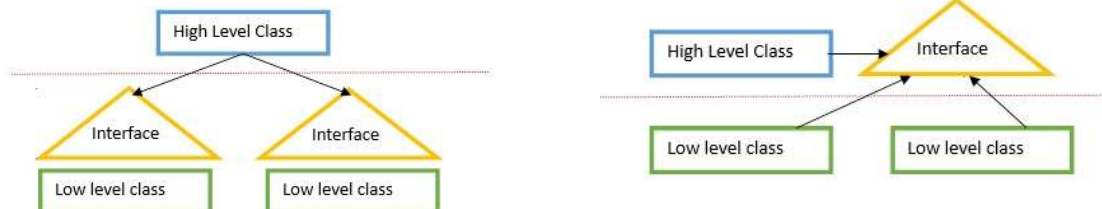
In anderen Worten

Dependency-Inversion-Prinzip

“A: High-level modules should not depend on low level modules. Both should depend on abstractions.

B: Abstractions should not depend upon details. Details should depend upon abstractions.”

– Robert C. Martin: The Dependency Inversion Principle



Damit ist sichergestellt, dass die Abhängigkeitsbeziehungen immer in eine Richtung verlaufen, von den konkreten zu den abstrakten Modulen, von den abgeleiteten Klassen zu den Basisklassen. Damit werden die Abhängigkeiten zwischen den Modulen reduziert und insbesondere zyklische Abhängigkeiten vermieden.

Wie solide ist SOLID

SOLID Prinzipien in der echten Welt

- Fragmentierung der Code-Base durch zu viele Mikro-Klassen (SRP, OCP, ISP)
- OCP mit zu vielen Abstraktionen vor eigentlicher Implementierung führen zu Overengineering (YAGNI)
- LSP: [Kreis-Ellipse-Problem](#) oder ein Pinguin als Vogel kann nicht fliegen
- ISP kann LSP Probleme lösen



Sobald wir SOLID in der echten Welt anwenden wollen, zeigen sich die ersten Risse.

Liegt es daran, dass wir SOLID nicht richtig verinnerlicht haben?

- * **SRP:** Fragmentierung der Code-Base mit zu vielen Micro-Classes
- * **Open / Close:** Zu viele Abstraktionen
- * **LSP:** Bird.fly() muss von Penguin implementiert werden, der nicht fliegen kann

<https://www.youtube.com/watch?v=V3TUEeB0kW0>

SOLID



SRP – Single Responsibility Principle



OCP – Open/Closed Principle



LSP – Liskovs Substitution Principle



ISP – Interface Segregation Principle



DIP – Dependency Inversion Principle



Warum ist das so?

Nach dem Lab (dem Artikel) werde ich das erklären.

Kritischer Blick

Wird Architektur überbewertet?



heise.de/blog/architektur-ist-ueberbewertet-und-was-wir... / 15min

Normalerweise würde ich an dieser Stelle eine Demo der SOLID Prinzipien machen.

Ist das für euch interessant?

Vorschlag: Lest euch diesen Artikel durch, lasst uns darüber diskutieren und danach entscheiden.

Architektur ist überbewertet

https://youtu.be/C7TMa_kYANA?t=387

Programmierparadigmen

Ausgeblendet, da nur ergänzend

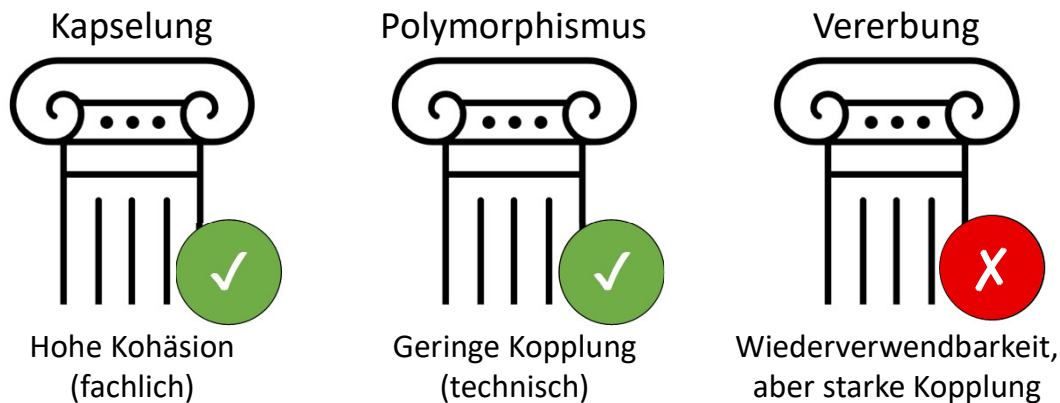
Es gibt verschiedene Methoden (Paradigmen), Zusammenhänge der realen Welt in ein Computerprogramm zu überführen. Moderne Programmiersprachen unterstützen oft mehrere davon, die sich kombinieren lassen. Man unterscheidet:

- **Imperativ:** Das Programm besteht aus einer Folge von Anweisungen, die es sequenziell ausführt. Im Zentrum dieses Paradigmas stehen Funktionen und Prozeduren; bekannte Vertreter dieses Sprachansatzes sind **C und Pascal**.
- **Objektbasiert:** Es gibt Objekte zum Kapseln von Daten und Funktionen, weiter gehende Konzepte wie Vererbung fehlen jedoch. Objektbasierte sind Vorstufen objektorientierter Sprachen. Frühere Versionen von **JavaScript** gehören dazu.
- ➔ **Objektorientiert:** Programmiersprachen dieser Gattung erweitern die objektbasierten um Konzepte wie Vererbung und Polymorphie. Typische Vertreter sind **Java, C# und C++**.
- **Funktional:** Ein funktionales Programm besteht aus einer Reihe von Funktionsaufrufen. Eigenständige Wertzuweisungen existieren nicht. Alle Elemente können als Funktionen aufgefasst werden. Typische Einsatzgebiete sind künstliche Intelligenz, Compilerbau sowie technische und mathematische Anwendungen. Wichtige funktionale Programmiersprachen sind **LISP** und **Haskell**.
- **Logisch:** Im Mittelpunkt stehen Fakten und Regeln. Fakten sind wahre Aussagen im Sinne der Logik. Im Fokus steht das Beschreiben des Problems, nicht der Lösungsalgorithmus. **Prolog** ist die bekannteste logische Programmiersprache.
- **Deklarativ:** Dabei handelt es sich um einen Überbegriff für das funktionale und logische Paradigma. Zentral ist das Beschreiben des gewünschten Ergebnisses, nicht der Weg dorthin. Die Structured Query Language (**SQL**) und andere Abfragesprachen folgen diesem Paradigma.

Quelle:

<https://www.heise.de/select/ix/2017/6/1495900368728612>

Die Drei Säulen der OOP



Versuchen wir einen anderen Blick auf das Problem zu werfen.

Was ist OOP und wie nennen sie?

--> Ein Paradigma, d. h. fundamentaler Programmierstil

--> **Imperatives** Paradigma: Struktur durch Schleifen, Entscheidungen und ggf. Funktionen (z. B. Basic, C)

(Weitere Paradigmen: Objektbasiert [JS], Funktional [LISP, Haskell], Logisch [Prolog], Deklarativ [SQL])

Was haltet ihr von den 3 Säulen der OOP? Gut / Schlecht?

1. Säule Kapselung

OOP führt Container ein (Objekt) wo inhaltlich zusammengehörende Daten **gekapselt** werden.

Prinzip der Kapselung bildet die Realität deutlich besser ab.

Beispiel Vektor mit Origin, Winkel, Länge und Operationen wie Drehung, Verschiebung.

Niemand sollte von außen Daten ändern können um Seiteneffekte zu verhindern.

Auch Funktionen, Komponenten und Services kapseln eine "Black Box".

Kapselung ist eine sehr gute Sache, weil es die Kohäsion fordert,

was wichtig für eine gute Struktur / Architektur ist.

Anm.: Aggregates und DDD basieren ebenfalls auf dem Konzept der Kapselung.

2. Säule Polymorphismus

Verschiedene Objekte stellen dieselbe Methode zur Verfügung, aber die Implementierung kann sich unterscheiden.

Client muss beim Aufruf keine Interna kennen.

Bsp. geom. Figuren GetArea() ohne zu wissen zu müssen um welche geom. Figur es sich handelt.

Selbes Prinzip gilt auch für eine API welche ein "Interface" implementiert.

Gleiches gilt für **überladene Funktionen**.

Es geht um Austauschbarkeit und Entkopplung von Interface vs. Implementierung.

Anm.: Duck Typing ist eine andere Art der Umsetzung von Polymorphie.

3. Konzept = Vererbung

Stärkstes Argument für Vererbung: Wiederverwendbarkeit / DRY

Problem: Es geht nicht mehr um erstrebenswerte Prinzipien wie Kohäsion / Kopplung

sondern pragmatische Aspekte wie weniger Code. Das ist eine ganz andere Qualität.

Ferner wird das Versprechen von DRY nicht eingehalten, sondern es entstehen **NEUE** Probleme. (nächste Folie)

Warum OOP überbewertet ist

<https://www.youtube.com/watch?v=zDx5d1tyoCM>

Probleme durch Vererbung

Neue Probleme statt "Wiederverwendbarkeit"

- Diskussion [Einfach- vs. Mehrfachvererbung](#)
- [Ko- und Kontravarianz](#)
- [Fragile Base Class Problem](#)
- [Kreis-Ellipse-Problem](#)
- usw.

Prinzip

Composition over Inheritance!

Eigentlich dienen die ganzen Prinzipien nur dazu, den **Scherbenhaufen** der Vererbung aufzuräumen.

Mit dem Prinzip "Composition over Inheritance" gibt es sogar ein Prinzip, dass sich **aktiv gegen** Vererbung ausspricht!

Grundsätzliche Frage der Architektur

Wo im Code sollte ich
mich um **was** kümmern?

Oder:

- Wer ist wofür Verantwortlich?
- Welche Komponente ist zuständig?

Im mathematischen Sinne gibt es keine eindeutige Antwort auf diese Fragen.

>> Am Ende des Tages müssen die getroffenen Entscheidungen ein höheres Ziel unterstützen:

Die Fachlichkeit!

- >> Deshalb entwickeln WIR Software überhaupt.
- >> Wir schreiben Software nicht als Selbstzweck sondern...
- >> ... als Mittel um fachlichen Zweck zu erfüllen.
- >> Daran muss sich Architektur messen lassen!

Architektur ist überbewertet

https://youtu.be/C7TMa_kYANA?t=488

Essenzielle "Axiome" guter Architektur

Unter dem Strich lässt sich folgern:

- ✓ Niedrige **technische** Kopplung
- ✓ Hohe **fachliche** Kohäsion
- ✓ Weniger ist mehr

Kapselung und Kohäsion gelten auf Ebene von Funktionen, Objekten, Komponenten, Services usw.

Vererbung wird völlig überbewertet und ist unwichtig.

Jede Abstraktion bringt auch eine Indirktion mit sich.

Weniger ist mehr (YAGNI)

- Flache Hierarchien
- Unnötige Abstraktionen vermeiden
- Komplexität reduzieren
- Technische Schuld abbauen



Kundensicht



Entwicklersicht